

BOOSTING ALGORITHM IN REGRESSION- ASSIGNMENT

➤ Regression:

Regression is one of the algorithm used to predict the “Number”.if your output will be in number means we can go for Regression.

➤ Boosting:

Boosting in regression is an ensemble learning technique where multiple weak models are built sequentially, and each new model corrects the errors of previous models to improve prediction accuracy for continuous numerical output.

Boosting means Many weak models join together to make one strong model.

➤ Why it is used?

- ✓ Without boosting: If we use only 1 model then accuracy will low and mistake will happen more.
- ✓ With boosting: improves accuracy, reduces error, better prediction

➤ How Boosting Works (Step by Step)

Step 1:

Build first simple model.

Step 2:

Find errors.

Step 3:

Next model focuses on previous errors.

Step 4:

Again next model correct remaining errors.

Step 5:

Combine all models. Final strong prediction.

Since output is number:

It tries to reduce the difference between actual value and predicted value and error

Example:

Actual = 50000, Predicted = 42000

Error = 8000

Next model tries to reduce this.

Boosting Algorithms for Regression

1. AdaBoost Regressor:

AdaBoost is a boosting algorithm that builds models sequentially, where each new model gives more importance to previously mispredicted samples to improve overall accuracy.

- ✓ **Full Form:** Adaptive Boosting
- ✓ **Type:** Ensemble / Boosting algorithm
- ✓ **Used for:** Classification and Regression
- ✓ **Works by:** Combining many weak learners (usually decision trees) sequentially
- ✓ **Main Idea:** Next model focuses more on previous model mistakes
- ✓ **Learns by:** Giving higher weight to wrongly predicted samples

SYNTAX:

```
from sklearn.ensemble import AdaBoostRegressor
from sklearn.tree import DecisionTreeRegressor
```

```
model = AdaBoostRegressor(
    estimator=DecisionTreeRegressor(max_depth=3),
    n_estimators=50,
    learning_rate=1.0,
    loss='linear',
    random_state=42
)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

2.XGBoost: XGBoost is an optimized gradient boosting algorithm where trees are built sequentially to minimize previous errors and improve prediction accuracy.

- **Full Form:** Extreme Gradient Boosting
- **Type:** Ensemble / Boosting algorithm
- **Used for:** Classification and Regression
- **Works by:** Combining many decision trees sequentially
- **Main Idea:** Each new tree corrects previous tree errors (residuals)
- **Learning Method:** Gradient boosting (minimizes loss function)

SYNTAX:

```
from xgboost import XGBRegressor
```

```
model = XGBRegressor(  
    n_estimators=100,  
    learning_rate=0.1,  
    max_depth=6,  
    random_state=42  
)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

Advantages

- High accuracy
- Fast performance
- Handles missing values
- Reduces overfitting using regularization
- Feature importance available
- Works well for structured/tabular data

3.LightGBM (LG Boost) :

LightGBM is a gradient boosting framework that builds trees leaf-wise to achieve faster training and high accuracy, especially on large datasets

- **Full Form:** Light Gradient Boosting Machine
- **Short Name:** LightGBM / LGBM
- **Type:** Ensemble / Boosting algorithm
- **Developed by:** Microsoft
- **Used for:** Classification and Regression

Working Principle

- Uses many decision trees sequentially
- Each new tree corrects previous tree errors
- Uses **leaf-wise tree growth** (faster than level-wise)

SYNTAX:

```
from lightgbm import LGBMRegressor
```

```
model = LGBMRegressor(  
    n_estimators=100,  
    learning_rate=0.1,  
    num_leaves=31,  
    random_state=42  
)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

Advantages

- High accuracy
- Fast performance
- Handles missing values
- Reduces overfitting using regularization
- Feature importance available
- Works well for structured/tabular data

Simply,

-  **AdaBoost** = Focus on mistakes
-  **XGBoost** = Smart error correction
-  **LightGBM** = Faster smart correction

When to Use

Use AdaBoost:

- Small dataset
- Simple project
- Easy explanation needed

Use XGBoost:

- Best overall performance
- High accuracy needed
- Popular in projects/interviews

Use LightGBM:

- Huge dataset
- Need fast training
- Low memory needed

Comparison of AdaBoost, XGBoost, LightGBM:

Feature	AdaBoost	XGBoost	LightGBM
Full Form	Adaptive Boosting	Extreme Gradient Boosting	Light Gradient Boosting Machine
Type	Boosting	Gradient Boosting	Gradient Boosting
Working Style	Focuses on wrong samples	Corrects residual errors	Corrects errors using leaf-wise growth
Speed	Medium	Fast	Very Fast
Accuracy	Good	Very High	Very High
Overfitting Control	Low	High (Regularization)	High
Missing Values	No strong support	Yes	Yes
Best For	Small clean data	Medium/Large structured data	Very large datasets
Memory Usage	Medium	Medium	Low
Training Time	Slower than XG/LGBM	Fast	Fastest
Complexity	Easy	Medium	Medium